

Moderni JavaScript za Node.js

Skripta i materijal za vježbu

Predmet:	Razvoj web aplikacija
Akadska godina:	2025/2026
Fakultet:	Prirodno-matematički fakultet
Univerzitet:	Univerzitet u Sarajevu

Šta ćemo savladati: razliku između starog (ES5) i modernog (ES6+) JavaScript-a, ključne sintakse koje se koriste u Express kodu, te osnovne koncepte Node.js okruženja.

Sadržaj

1. Uvod: šta je "moderni" JavaScript?
2. var / let / const
3. Arrow funkcije (=>)
4. Template literali (`backticks` i `\${...})
5. Destruktuiranje objekata
6. Destruktuiranje nizova
7. Spread (...) i Rest (...)
8. Skraćeni objekt zapis (shorthand) i default parametri
9. Optional chaining (?.) i nullish coalescing (??)
10. Array metode: map, filter, reduce
11. Array metode: find, some, every, includes
12. Šta je Node.js? Razlika u odnosu na browser
13. Moduli: CommonJS (require / module.exports)
14. Moduli: ES Modules (import / export)
15. Ugrađeni moduli: fs (rad sa fajlovima)
16. Ugrađeni moduli: path
17. Ugrađeni moduli: crypto
18. Objekat process: argv, env, cwd, exit
19. Asinhroni JS: zašto callbacks?
20. Promise — osnove
21. async / await
22. try / catch sa async funkcijama
23. fetch u Node 18+
24. npm i package.json: zavisnosti i skripte
25. Praktični tipovi (node --watch, REPL, .env)
26. Završne misli i šta dalje

Uvod: šta je "moderni" JavaScript?

JavaScript je jezik koji se mijenjao tokom godina. Svaka godina donosi nove verzije standarda (ECMAScript, kraće **ES**). Stari kod koji vidimo na starijim tutorijalima koristi **ES5** (iz 2009. godine). Sve od 2015. godine je "moderni" JavaScript: **ES6 (ES2015)**, **ES2016**, **ES2017**... pa sve do današnjih verzija.

Moderni JavaScript donosi **kraću, jasniju i sigurniju** sintaksu. Na ovim vježbama (i u Express kodu) susrećemo se sa modernim JS-om skoro u svakoj liniji koda. Bez razumijevanja ovih osnovnih konstrukcija, kod izgleda kao "magija".

Šta ćemo pokriti u ovoj skripti

- Sintaksu koja se najčešće vidi: **let/const**, **arrow**, **destruktuiranje**, **spread/rest**.
- Asinhrono programiranje: **Promise** i **async/await**.
- Osnove Node.js okruženja: **moduli**, **ugrađene biblioteke**, **process**.
- Praktične alate koji štede vrijeme.

Savjet

Nakon svake teme probaj primjer u terminalu. Pokreni node bez argumenata da otvoriš REPL, ili kreiraj fajl `test.js` i pokreni `node test.js`. Učenje "rukama" je deset puta brže od čitanja.

var / let / const

Prije ES6, jedina riječ za deklaraciju varijable bila je **var**. Od ES6 imamo dvije nove: **let** i **const**. One se ponašaju ozbiljnije i predvidljivije.

Stari način (var)

```
var ime = "Ana";
var ime = "Marko";    // ovo je OK – var dozvoljava redeklaraciju
console.log(ime);      // "Marko"

for (var i = 0; i < 3; i++) {
  // ...
}
console.log(i);        // 3 – var "curi" izvan bloka!
```

Moderni način (let / const)

```
let ime = "Ana";
ime = "Marko";        // OK, vrijednost se može mijenjati
// let ime = "Lana";  // GRESKA: ne može redeklaracija

const PI = 3.14;
// PI = 3.15;         // GRESKA: const se ne mijenja

for (let i = 0; i < 3; i++) {
  // ...
}
// console.log(i);    // GRESKA: i postoji SAMO unutar for petlje
```

Pravilo palca

- Koristi **const** za sve dok ti ne treba mijenjanje.
- Koristi **let** kad stvarno mijenjas vrijednost (npr. brojac).
- **var** izbjegavaj u novom kodu.

Napomena

const NE znaci "sve zamrznuto". Znacilo samo da se **varijabla ne može prevezati** na drugu vrijednost. Ako je `const lista = []`, mozes da radis `lista.push(1)` bez problema — samo ne mozes da kazes `lista = [1, 2]`.

Arrow funkcije (=>)

Arrow funkcije su kraći način pisanja funkcija. Toliko često ih vidimo u Express kodu da je nemoguće raditi bez razumijevanja ove sintakse.

Klasična funkcija (stari način)

```
function kvadrat(x) {  
  return x * x;  
}  
  
router.get("/", function (req, res) {  
  res.send("Pozdrav!");  
});
```

Arrow funkcija (moderni način)

```
const kvadrat = (x) => {  
  return x * x;  
};  
  
// jos krace – ako ima samo "return", uvijek zagrada:  
const kvadrat2 = (x) => x * x;  
  
// jedan parametar – moze i bez zagrada:  
const kvadrat3 = x => x * x;  
  
router.get("/", (req, res) => {  
  res.send("Pozdrav!");  
});
```

Kada koristiti zagrade za parametre?

```
// nula parametara – OBAVEZNE prazne zagrade  
const sada = () => new Date();  
  
// jedan parametar – zagrade su opcione  
const dupli = x => x * 2;  
const dupli2 = (x) => x * 2;  
  
// dva ili vise – OBAVEZNE zagrade  
const zbir = (a, b) => a + b;
```

Pažnja: this

Arrow funkcije **nemaju svoj this**. Naslijeđuju ga iz okruženja gdje su definisane. Ovo je obično dobro, ali znaci da arrow ne treba koristiti kao metodu objekta ako trebas pristup `this.svojstvo`.

Template literali

Stari način spajanja stringa i varijabli bio je gnjavaza: stalno + i navodnici. Moderni JS uvodi **template literale** sa backtick znacima (```) i `${...}` sintaksom za umetanje vrijednosti.

Stari način

```
var ime = "Ana";
var godina = 23;
var poruka = "Zdravo " + ime + "! Ti imas " + godina + " godina.";

var html = "<div class=\"card\">\n" +
    "  <h2>" + ime + "</h2>\n" +
    "</div>";
```

Moderni način

```
const ime = "Ana";
const godina = 23;
const poruka = `Zdravo ${ime}! Ti imas ${godina} godina.`;

// Višekratni string ide BEZ spajanja, samo unutar backtick-a:
const html = `

## Gdje ćeš ovo vidati



- Express rute: res.redirect(`/studenti/${id}`)
- SQL upiti: db.prepare(`SELECT * FROM student WHERE id = ?`)
- Logovi: console.log(`Server slusa na portu ${port}`)



### Savjet



Backtick ( ` ) je tipka iznad TAB-a, lijevo od "1". Na BHS rasporedu može tražiti AltGr ili dvostruki pritisak.



Moderni JS za Node.js



— 6 —



RWA · PMF Sarajevo


```

Destrukturiranje objekata

Destrukturiranje znaci "izvuci vise polja iz objekta u jednoj liniji". Vrlo cesto koristimo kod citanja `req.body` u Express rutama.

Stari način

```
router.post("/login", function (req, res) {  
  var korisnicko_ime = req.body.korisnicko_ime;  
  var lozinka = req.body.lozinka;  
  // dalje koristimo korisnicko_ime i lozinka...  
});
```

Moderni način

```
router.post("/login", (req, res) => {  
  const { korisnicko_ime, lozinka } = req.body;  
  // dalje koristimo korisnicko_ime i lozinka...  
});  
  
// Sa preimenovanjem:  
const { korisnicko_ime: username, lozinka: password } = req.body;  
  
// Sa default vrijednoscu:  
const { godine = 18 } = req.body;  
// ako req.body.godine ne postoji, godine = 18
```

Ugnijezdjeni objekti

```
const korisnik = {  
  ime: "Ana",  
  adresa: { grad: "Sarajevo", postanski: "71000" }  
};  
  
const { ime, adresa: { grad } } = korisnik;  
console.log(ime); // "Ana"  
console.log(grad); // "Sarajevo"
```

Zašto je ovo važno

U Express i Node.js kodu skoro svaka funkcija prima objekat (npr. `req`, `options`, `config`). Destrukturiranje cini kod 50% kraćim.

Destrukturiranje nizova

Slično kao kod objekata, ali za nizove. Razlika: kod nizova bitan je **redoslijed**, a ne ime polja.

Primjer

```
const boje = ["crvena", "zelena", "plava"];

// stari nacin:
// var prva = boje[0];
// var druga = boje[1];

// moderni nacin:
const [prva, druga, treca] = boje;
console.log(prva); // "crvena"
console.log(treca); // "plava"

// Mogu i preskoci elemente:
const [, , samo_treca] = boje;
console.log(samo_treca); // "plava"

// Sa default vrijednoscu:
const [a = 1, b = 2, c = 3, d = 4] = [10, 20];
console.log(a, b, c, d); // 10 20 3 4
```

Klasičan primjer: zamjena dvije varijable

```
let x = 1, y = 2;
[x, y] = [y, x];
console.log(x, y); // 2 1
```

Iz funkcije koja vraća više vrijednosti

```
function podijeli(a, b) {
  return [Math.floor(a / b), a % b];
}

const [kolicnik, ostatak] = podijeli(10, 3);
console.log(kolicnik); // 3
console.log(ostatak); // 1
```


Spread (...) i Rest (...)

Tri tačke ... imaju dvije namjene: **spread** (rasipanje) i **rest** (skupljanje). Sintaksa je ista, kontekst određuje šta radi.

Spread: rasipanje elemenata

```
const niz1 = [1, 2, 3];
const niz2 = [4, 5, 6];

// Spajanje nizova:
const sve = [...niz1, ...niz2];
// [1, 2, 3, 4, 5, 6]

// Kopija niza (NOVI niz, ne referenca):
const kopija = [...niz1];

// Isti princip za objekte:
const osnovne = { ime: "Ana", godine: 23 };
const sa_emailom = { ...osnovne, email: "ana@pmf.unsa.ba" };
// { ime: "Ana", godine: 23, email: "ana@pmf.unsa.ba" }

// Override polja – pravilo: posljednje pobjeđuje
const azurirano = { ...osnovne, godine: 24 };
// { ime: "Ana", godine: 24 }
```

Rest: skupljanje viška argumenata

```
// U funkciji – sve sto dodje "viska" stavi u jedan niz:
function zbir(...brojevi) {
  return brojevi.reduce((a, b) => a + b, 0);
}
zbir(1, 2, 3, 4); // 10

// U destrukuiranju – "ostatak" objekta:
const korisnik = { ime: "Ana", godine: 23, email: "a@b.ba", grad: "Tuzla" };
const { ime, ...ostalo } = korisnik;
console.log(ime); // "Ana"
console.log(ostalo); // { godine: 23, email: "a@b.ba", grad: "Tuzla" }
```

Često korišten trik

Kada želiš "dodati polje u objekat" bez mutiranja originala, koristi spread: `const novi = { ...stari, novoPolje: 5 }`; Originalni objekat ostaje netaknut.

Skraćeni objekt zapis i default parametri

Shorthand: kraći zapis objekta

Kada varijabla i polje imaju **isto ime**, ne moramo pisati dvaput.

```
// stari nacin:
function napraviStudenta(ime, prezime, email) {
  return {
    ime: ime,
    prezime: prezime,
    email: email
  };
}

// moderni:
function napraviStudenta(ime, prezime, email) {
  return { ime, prezime, email };
}

// Vidjeces stalno ovo u Express rutama:
res.render("student-forma", { student, grupe, greska });
// (ekvivalentno: { student: student, grupe: grupe, greska: greska })
```

Default parametri funkcije

```
// stari nacin:
function pozdrav(ime) {
  ime = ime || "gost"; // ako je undefined, koristi "gost"
  return "Zdravo, " + ime;
}

// moderni:
function pozdrav(ime = "gost") {
  return `Zdravo, ${ime}`;
}

pozdrav(); // "Zdravo, gost"
pozdrav("Ana"); // "Zdravo, Ana"

// Sa Express upitima:
function listaj(limit = 10, sortBy = "ime") {
  return db.prepare(`SELECT * FROM student ORDER BY ${sortBy} LIMIT ?`).all(limit);
}
```

Skraćene metode unutar objekta

```
// stari:
const api = {
  poshalji: function (data) { /* ... */ },
  primi: function () { /* ... */ }
};
```

```
// moderni:
const api2 = {
  poshalji(data) { /* ... */ },
  primi() { /* ... */ }
};
```

Optional chaining (?.) i nullish coalescing (??)

Dvije relativno nove (2020+) ali izuzetno korisne sintakse. Štedi puno `if` provjera.

Optional chaining ?.

Kada pristupamo svojstvu kroz lanac (`a.b.c.d`) i bilo koje od polja može biti `null/undefined`, kod pukne. `?.` pita: "ako je lijevo `null/undefined`, vrati `undefined` umjesto da pucaš".

```
// stari nacin (gnjavaza):
var grad = null;
if (korisnik && korisnik.adresa && korisnik.adresa.grad) {
  grad = korisnik.adresa.grad;
}

// moderni:
const grad = korisnik?.adresa?.grad;
// ako bilo gdje na putu naidje na null/undefined,
// `grad` je undefined umjesto da padne kod.

// Radi i sa pozivima funkcija:
const rezultat = obj.mozdaPostoji?.();

// I sa pristupom po indeksu:
const prvi = niz?.[0];
```

Nullish coalescing ??

Stari operator `||` vraća desnu stranu ako je lijeva "falsy" (npr. 0, prazan string, false). To je nekad krivo: ako je vrijednost legitimno 0, stari kod ga zamijeni. `??` reagira **samo** na `null` i `undefined`.

```
// stari nacin:
var godine = korisnik.godine || 18;
// PROBLEM: ako su godine = 0, koristit ce 18!

// moderni:
const godine = korisnik.godine ?? 18;
// godine = 0 -> ostaje 0
// godine = null/undefined -> postaje 18

// Kombinacija sa optional chaining:
const grad = korisnik?.adresa?.grad ?? "Nepoznato";
```

Kada koji?

`?.` za sigurno čitanje kroz lanac. `??` za fallback samo kad je `null/undefined`. `||` samo kad zaista hoćeš da reagira i na `0/"/false`.

Array metode: map, filter, reduce

Najmoćniji alat modernog JS-a. Pisanje petlji se uglavnom zamjenjuje ovim metodama jer je rezultat čitljiviji.

map: transformiše svaki element

```
const brojevi = [1, 2, 3, 4];

// stari nacin:
const kvadrati_stari = [];
for (let i = 0; i < brojevi.length; i++) {
  kvadrati_stari.push(brojevi[i] * brojevi[i]);
}

// moderni:
const kvadrati = brojevi.map(x => x * x);
// [1, 4, 9, 16]

// Iz baze: dohvaceni redovi -> drugaciji format
const imena = studenti.map(s => `${s.ime} ${s.prezime}`);
```

filter: zadrži samo elemente koji prolaze provjeru

```
const brojevi = [1, 2, 3, 4, 5, 6];

const parni = brojevi.filter(x => x % 2 === 0);
// [2, 4, 6]

// Iz baze: filtriraj klijentski (kad ima malo podataka)
const polozeni = upisi.filter(u => u.ocjena >= 6);

// Kombinacija sa map (chain):
const imenaPolozenih = upisi
  .filter(u => u.ocjena >= 6)
  .map(u => u.student_ime);
```

reduce: skupi niz u jednu vrijednost

```
const brojevi = [1, 2, 3, 4];

// suma:
const ukupno = brojevi.reduce((akumulator, trenutni) => akumulator + trenutni, 0);
// 10
// 0 = pocetna vrijednost akumulatora

// prosjek ocjena:
const ocjene = [7, 8, 9, 10, 6];
const prosjek = ocjene.reduce((s, o) => s + o, 0) / ocjene.length;

// Grupisanje:
const studenti = [
```

```
{ ime: "Ana", grad: "Sarajevo" },
{ ime: "Marko", grad: "Tuzla" },
{ ime: "Lana", grad: "Sarajevo" },
];

const poGradovima = studenti.reduce((acc, s) => {
  if (!acc[s.grad]) acc[s.grad] = [];
  acc[s.grad].push(s.ime);
  return acc;
}, {});
// { Sarajevo: ["Ana", "Lana"], Tuzla: ["Marko"] }
```

find, some, every, includes

find: vraća PRVI element koji prolazi uslov

```
const studenti = [
  { id: 1, ime: "Ana" },
  { id: 2, ime: "Marko" },
  { id: 3, ime: "Lana" },
];

const trazeni = studenti.find(s => s.id === 2);
// { id: 2, ime: "Marko" }

const nepostojeci = studenti.find(s => s.id === 99);
// undefined
```

some: postoji li bar jedan koji prolazi?

```
const ocjene = [5, 7, 9, 10];

const imaPet = ocjene.some(o => o === 5); // true
const imaJedinicu = ocjene.some(o => o === 1); // false
```

every: PROLAZE li SVI?

```
const sviPolozili = ocjene.every(o => o >= 6); // false (zbog 5)
const sviUOpsegu = ocjene.every(o => o >= 1 && o <= 10); // true
```

includes: postoji li tacno ova vrijednost?

```
const boje = ["crvena", "plava", "zuta"];

boje.includes("plava"); // true
boje.includes("zelena"); // false

// Radi i sa primitivnim brojevima:
[1, 2, 3].includes(2); // true

// Za objekte NE RADI (poredi po referenci)!
const a = { x: 1 };
[{ x: 1 }].includes(a); // false, jer to NIJE isti objekat
```

Pravilo

Kad samo trebaš znati "ima li" → **some** ili **includes**. Kad ti treba sam element → **find**. Kad transformišiš svaki → **map**. Kad smanjuješ broj elemenata → **filter**.

Šta je Node.js? Razlika u odnosu na browser

JavaScript je nastao 1995. da bi se izvršavao u browseru — da animira klikove, validira forme, itd. **Node.js** (2009) uzima isti jezik i pokreće ga **izvan** browsera. Time JavaScript postaje jezik za server, alate i skripte.

Šta Node.js ima što browser nema

- Pristup **fajlovima** na disku (modul `fs`).
- Pristup **mreži** kao server (modul `http`).
- Pristup **procesu** i okruženju (`process.env`, `process.argv`).
- Mogućnost **instaliranja paketa** sa `npm`-a.
- Kriptografiju, streaming, child procese...

Šta NEMA u Node.js (a ima u browseru)

- Nema `window` objekta.
- Nema `document` objekta (DOM-a).
- Nema `localStorage` / `cookie` u browser smislu.
- Nema `alert`, `confirm`, `prompt`.

Šta dijele

- Sav osnovni jezik: `const`, `let`, `if`, `for`, funkcije, objekti, nizovi, `Date`, `Math`...
- Sve moderne sintakse iz prethodnih poglavlja.
- `fetch` (u Node 18+).

Praktično

Kod u tvojim vježbama (Express rute) izvršava se u Node.js. Kod u `<script>` tagu u EJS view-u izvršava se u browseru. Iako je jezik isti, dostupni objekti su različiti.

Moduli: CommonJS (require / module.exports)

U Node.js, svaki .js fajl je svoj **modul**. Varijable definisane u jednom fajlu nisu vidljive drugom — moramo ih eksplicitno izvesti (**export**) i učitati (**require**). CommonJS je "klasični" način koji koristimo u svim vježbama.

Izvoz: module.exports

```
// fajl: matematika.js

function zbir(a, b) {
  return a + b;
}

function razlika(a, b) {
  return a - b;
}

// Nacin 1 – izvezi vise stvari kao objekat:
module.exports = { zbir, razlika };

// Nacin 2 – izvezi samo jednu stvar:
// module.exports = zbir;
```

Uvoz: require

```
// fajl: app.js

// Ucitaj cijeli objekat:
const matematika = require("./matematika");
console.log(matematika.zbir(2, 3));

// Ili odmah destrukuiraj:
const { zbir, razlika } = require("./matematika");
console.log(zbir(2, 3));      // 5
console.log(razlika(5, 2));   // 3
```

Putanje: relativne i pakete

```
// Vlastiti fajlovi – relativne putanje (POCINJU sa ./ ili ../)
const db = require("./db");
const userRouter = require("./routes/users");
const helper = require("../utils/helper");

// npm paketi – samo ime paketa, BEZ ./ na pocetku
const express = require("express");
const path = require("path");      // ugradjen u Node, bez install

// Node ce ih traziti u node_modules/
```

Šta vidiš u vježbama

Na vrhu svakog Express fajla: `const express = require("express");` — uvozi npm paket. `const db = require("./db");` — uvozi tvoj vlastiti fajl `db.js`.

Moduli: ES Modules (import / export)

ES Modules je **noviji** standard koji dolazi iz browser svijeta. Node ga podržava od verzije 14+. Sintaksa je drugačija od CommonJS-a.

Na našim vježbama koristimo CommonJS (require), ali dobro je prepoznati ES sintaksu jer je vidiš u dokumentaciji i tutorijalima.

Izvoz

```
// matematika.js

// Imenovani izvoz:
export function zbir(a, b) {
  return a + b;
}

export const PI = 3.14;

// Default izvoz (samo jedan po fajlu):
export default function vaznaFunkcija() { /* ... */ }
```

Uvoz

```
// Imenovani uvoz – moras znati taca imena:
import { zbir, PI } from "./matematika.js";

// Default uvoz – biras ime:
import vazna from "./matematika.js";

// Oboje:
import vazna, { zbir, PI } from "./matematika.js";

// Sve odjednom kao namespace:
import * as M from "./matematika.js";
console.log(M.zbir(1, 2));
```

Kako uključiti ES Module u Node?

- Dodaj u package.json: "type": "module"
- Tada su sve .js datoteke ES Module.
- Alternativa: imenuj fajlove sa .mjs ekstenzijom.

Za naše vježbe

Ostajemo na CommonJS (require) jer to koristi Express generator i većina tutorijala. ES Modules je dobro znati prepoznati, ali ne moramo prelaziti.

Ugrađeni moduli: fs (File System)

Modul **fs** služi za rad sa fajlovima i folderima. Dolazi sa Node-om, ne treba ga instalirati.

Čitanje fajla (sinhrono — za skripte)

```
const fs = require("fs");

// Sinhrona verzija (jednostavna, blokira tok):
const tekst = fs.readFileSync("podaci.txt", "utf8");
console.log(tekst);

// "utf8" znaci "vрати mi string". Bez tog argumenta, dobio bi
// Buffer (binarni podaci) – sto rijetko trebamo za tekst.
```

Čitanje sa async/await (preporučeno za server)

```
const fs = require("fs/promises"); // promise-based verzija

async function ucitaj() {
  const tekst = await fs.readFile("podaci.txt", "utf8");
  console.log(tekst);
}

ucitaj();
```

Pisanje fajla

```
const fs = require("fs");

// Sinhrono:
fs.writeFileSync("rezultat.txt", "Zdravo svijete!", "utf8");

// Dopuna na kraj (append):
fs.appendFileSync("log.txt", `[${new Date().toISOString()}] Nova prijava\n`);

// JSON:
const podaci = { ime: "Ana", godine: 23 };
fs.writeFileSync("korisnik.json", JSON.stringify(podaci, null, 2));
```

Provjera postojanja i listanje

```
// Provjeri da li fajl postoji:
if (fs.existsSync("podaci.txt")) {
  console.log("Postoji!");
}

// Listanje foldera:
const fajlovi = fs.readdirSync("./public");
console.log(fajlovi);
// ["images", "javascripts", "stylesheets"]
```

Sinhrono ili asinhrono?

Za **jednokratne skripte** (one-off CLI alate, seed skripte) — slobodno koristi `readFileSync`. Za **Express rute** koje opslužuju mnogo korisnika — koristi asinhronu verziju (`fs/promises` + `await`) da ne blokiraš server.

Ugrađeni moduli: path

Modul **path** služi za rad sa **putanjama do fajlova**. Različiti operativni sistemi koriste različite separatore (Windows: \, Linux/Mac: /), i path nam to apstrahuje.

Spajanje putanja: path.join

```
const path = require("path");

// Bez modula path:
// "C:\\Users\\eldin\\projekat" + "\\public" + "\\style.css"
// Ovo je krhko i ne radi na Linuxu.

// Sa modulom path:
const putanja = path.join("public", "stylesheets", "style.css");
// Na Windowsu: "public\\stylesheets\\style.css"
// Na Linuxu: "public/stylesheets/style.css"

// __dirname = folder u kojem je trenutni fajl (apsolutna putanja)
const fullPath = path.join(__dirname, "..", "skola.db");
// Vodi nas jedan nivo iznad ovog fajla, u skola.db
```

Druge korisne funkcije

```
path.extname("slika.jpg");           // ".jpg"
path.basename("/a/b/slika.jpg");     // "slika.jpg"
path.dirname("/a/b/slika.jpg");       // "/a/b"
path.resolve("podaci.txt");           // apsolutna putanja od trenutnog dir-a
```

Gdje ovo vidiš u Express kodu

```
// app.js iz vježbi:
app.set("views", path.join(__dirname, "views"));
app.use(express.static(path.join(__dirname, "public")));

// db.js iz vježbi:
const dbPath = path.join(__dirname, "..", "skola.db");
```

Pravilo

Nikad ne piši putanje ručno sa / ili \. Uvijek koristi `path.join` ili `path.resolve`. Kod ti tada radi na svakom OS-u.

Ugrađeni moduli: crypto

Modul **crypto** daje kriptografske funkcije: hash, slučajni bajtovi, generisanje UUID-a, šifriranje. Dolazi sa Node-om.

Slučajan token (npr. za share link)

```
const crypto = require("crypto");

// Slučajnih 16 bajta -> 32 hex znaka:
const token = crypto.randomBytes(16).toString("hex");
console.log(token);
// "a3f9c1b2d8e7f0..." (32 znaka)

// UUID v4 (univerzalno jedinstven ID):
const id = crypto.randomUUID();
console.log(id);
// "550e8400-e29b-41d4-a716-446655440000"
```

Hash (jednosmjerni)

```
const crypto = require("crypto");

// SHA-256 hash stringa:
const hash = crypto
  .createHash("sha256")
  .update("moja-lozinka")
  .digest("hex");
console.log(hash);
// (uvijek isti za isti ulaz)
```

Napomena o lozinkama

Za **lozinke korisnika** ne koristimo `crypto.createHash` direktno. Koristimo **bcrypt** (kao u v10) jer dodaje "salt" i sporiji je — što je sigurnije protiv napadača. `crypto.randomBytes` je super za **tokene** (reset lozinke, share linkovi, session ID), ali za hash lozinke koristi `bcrypt`.

Kratko

crypto.randomBytes / **randomUUID** — slučajni nasumični ID-evi (sigurno, ne pogađaju se).
crypto.createHash — checksum tipa SHA-256. **bcrypt** (npm paket) — za lozinke.

Objekat process: argv, env, cwd, exit

process je globalni objekat u Node-u — dostupan svugdje bez require-a. Predstavlja trenutni Node proces. Najčešće ga koristimo za: argumente komandne linije, environment varijable, izlaz iz programa.

process.argv: argumenti komandne linije

```
// fajl: pozdrav.js
const args = process.argv;
console.log(args);
// Ako pokrenes: node pozdrav.js Ana 23
// Ispise:
// [
//   "/usr/bin/node",      <- putanja do node-a
//   "../pozdrav.js",      <- putanja do skripte
//   "Ana",                 <- nas argument 1
//   "23"                   <- nas argument 2
// ]

// Bolji nacin – preskoci prva dva:
const [ime, godine] = process.argv.slice(2);
console.log(`Zdravo ${ime}, imas ${godine} godina.`);
```

process.env: environment varijable

```
// Set u Linuxu/Mac: PORT=4000 node server.js
// Set u PowerShellu: $env:PORT=4000; node server.js

const port = process.env.PORT || 3000;
const dbHost = process.env.DB_HOST || "localhost";

console.log(`Server slusa na portu ${port}`);

// VAZNO: tajne (lozinke baze, API kljuceve, secret za sesije)
// NIKAD ne pisi u kod. Stavi u .env fajl + ucitaj sa dotenv paketom.
```

process.cwd() i process.exit()

```
// cwd = current working directory
// Folder iz kojeg je node POKRENUT (ne nuzno gdje je fajl!)
console.log(process.cwd());

// Izlaz iz programa:
if (greska) {
  console.error("Nije uspjelo!");
  process.exit(1); // 1 = greska, 0 = OK
}
```


Često korišten obrazac

U `bin/www` fajlu iz Express generatora vidiš `process.env.PORT || "3000"`. Ovo znači: ako je server hostovan (npr. Heroku), uzmi port koji daje, inače koristi 3000.

Asinhroni JavaScript: zašto callbacks?

Neke operacije **traju**: čitanje fajla, slanje HTTP zahtjeva, upit ka bazi. JavaScript je **jednonitni** — ne može stati i čekati, inače bi cijeli server stao. Zato koristi **asinhroni model**: započni operaciju, registruj šta da uradi kad završi, idi dalje.

Stari način: callbacks

```
const fs = require("fs");

// Drugi argument je FUNKCIJA koja se zove KAD ZAVRSI citanje:
fs.readFile("podaci.txt", "utf8", function (err, tekst) {
  if (err) {
    console.error("Greska:", err);
    return;
  }
  console.log("Procitao sam:", tekst);
});

console.log("Ovo ce se ispisati PRIJE 'Procitao sam:!'");
```

Problem: ugniježđeni callbacks ("callback hell")

```
// Kad jedna operacija ovisi o drugoj, ugniježdjivanje postaje pakao:
fs.readFile("a.txt", "utf8", function (err, a) {
  fs.readFile("b.txt", "utf8", function (err, b) {
    fs.readFile("c.txt", "utf8", function (err, c) {
      fs.writeFile("rezultat.txt", a + b + c, function (err) {
        // ... i tako u nedogled
      });
    });
  });
});

// Kod sve uvucen udesno -> "piramida propasti".
```

Zato je JS dobio **Promise** (poglavlje 20) i **async/await** (poglavlje 21). To su moderni načini da pišemo asinhroni kod kao da je sinhroni.

Promise — osnove

Promise (obećanje) je objekat koji predstavlja **buduću** vrijednost. Tri stanja: *pending* (čeka), *fulfilled* (uspjelo), *rejected* (greška). Promise je 90% temelj za `async/await` — pa ga važno razumjeti.

Korištenje već postojećeg Promise-a (`.then` / `.catch`)

```
const fs = require("fs/promises"); // promise verzija

fs.readFile("podaci.txt", "utf8")
  .then(tekst => {
    console.log("Procitao:", tekst);
    return tekst.toUpperCase();
  })
  .then(veliko => {
    console.log("Velikim:", veliko);
  })
  .catch(err => {
    console.error("Greska:", err);
  });
```

Lančanje (chaining)

```
// Svaki .then dobija ono sto je prethodni vratio:
fetch("https://api.example.com/users/1")
  .then(response => response.json())
  .then(user => {
    console.log(user.name);
    return fetch(`https://api.example.com/users/${user.id}/posts`);
  })
  .then(response => response.json())
  .then(posts => console.log(posts))
  .catch(err => console.error(err));
```

Promise.all: cekaj VISE Promise-a paralelno

```
const fs = require("fs/promises");

Promise.all([
  fs.readFile("a.txt", "utf8"),
  fs.readFile("b.txt", "utf8"),
  fs.readFile("c.txt", "utf8"),
])
  .then([a, b, c] => {
    console.log("Sva tri procitana!");
    console.log(a, b, c);
  })
  .catch(err => console.error("Bar jedan je pao:", err));
```

Razlika `.then` i `await`

`.then` i `await` rade istu stvar — ali `await` izgleda kao običan kod. Vidjet ćeš u sljedećem poglavlju.

async / await

async/await je "šećer" iznad Promise-a — piše se kao sinhroni kod, a u pozadini radi asinhrono. Najlakši i najčitljiviji način za rad sa asinhronim operacijama.

Pravilo: await samo unutar async funkcije

```
const fs = require("fs/promises");

// async pred function – sad mozes koristiti await unutra
async function ucitajPodatke() {
  const a = await fs.readFile("a.txt", "utf8");
  const b = await fs.readFile("b.txt", "utf8");
  const c = await fs.readFile("c.txt", "utf8");
  return a + b + c;
}

// Pozivanje (rezultat je opet Promise!):
ucitajPodatke().then(rez => console.log(rez));

// Ili iz druge async funkcije:
async function main() {
  const rez = await ucitajPodatke();
  console.log(rez);
}
main();
```

Sa arrow funkcijom

```
const ucitaj = async (putanja) => {
  return await fs.readFile(putanja, "utf8");
};

// Iz Express rute:
app.get("/podaci", async (req, res) => {
  const tekst = await fs.readFile("podaci.txt", "utf8");
  res.send(tekst);
});
```

Paralelno izvršavanje sa Promise.all

```
async function paralelno() {
  // Pokreni sve odjednom, cekaj sve da zavrse:
  const [a, b, c] = await Promise.all([
    fs.readFile("a.txt", "utf8"),
    fs.readFile("b.txt", "utf8"),
    fs.readFile("c.txt", "utf8"),
  ]);
  console.log(a, b, c);
}
```

Sinhrono vs asinhrono

await ne "blokira" cijeli server — samo PAUZIRA tu funkciju. Druge konekcije (drugi korisnici) i dalje rade. Zato je server brz.

try / catch sa async funkcijama

Kada koristimo `await`, greške se hvataju klasičnim **try/catch** blokom — kao u sinhronom kodu.

Bez try/catch — greška ruši rutu

```
app.get("/podaci", async (req, res) => {
  const tekst = await fs.readFile("ne-postoji.txt", "utf8");
  res.send(tekst);
});

// Ako fajl ne postoji, await baci gresku. Bez try/catch,
// Express vidi unhandled rejection i moze vratiti 500.
```

Sa try/catch — gracefully obradjujemo gresku

```
app.get("/podaci", async (req, res) => {
  try {
    const tekst = await fs.readFile("podaci.txt", "utf8");
    res.send(tekst);
  } catch (err) {
    console.error("Greska pri citanju:", err.message);
    res.status(500).send("Nesto je poslo po zlu.");
  }
});
```

Više await-a u jednom try blok-u

```
async function obradi(id) {
  try {
    const korisnik = await dohvatiKorisnika(id);
    const profil = await dohvatiProfil(korisnik.id);
    const slike = await dohvatiSlike(profil.id);
    return { korisnik, profil, slike };
  } catch (err) {
    // Bilo koji od tri await-a moze baciti gresku,
    // svi se hvataju ovdje.
    console.error("Greska negdje:", err);
    return null;
  }
}
```

Express + async

Ako se `async` ruta sruši bez `try/catch`, novije verzije Express-a (5+) automatski to obrade. Stariji Express 4 zahtijeva ručno hvatanje. Za sigurnost, **uvijek** stavi `try/catch` u `async` rutama.

fetch u Node 18+

Do Node 18, da bismo iz Node-a pozvali neki HTTP API, morali smo instalirati `axios` ili `node-fetch`. Od Node 18, **fetch** je ugrađen — isti kao u browseru.

GET zahtjev

```
// Bez instaliranja, samo direktno:
async function dohvatiPodatke() {
  const odgovor = await fetch("https://api.github.com/users/octocat");
  const podaci = await odgovor.json();
  console.log(podaci.name, podaci.bio);
}

dohvatiPodatke();
```

POST zahtjev

```
async function postaj() {
  const odgovor = await fetch("https://api.example.com/users", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ ime: "Ana", email: "ana@pmf.unsa.ba" })
  });

  if (!odgovor.ok) {
    throw new Error(`HTTP ${odgovor.status}`);
  }

  const novi = await odgovor.json();
  console.log("Kreiran:", novi);
}
```

Praktičan primjer — pozivanje javnog API-ja iz Express rute

```
app.get("/sala", async (req, res) => {
  try {
    const odgovor = await fetch("https://icanhazdadjoke.com/", {
      headers: { Accept: "application/json" }
    });
    const data = await odgovor.json();
    res.json({ sala: data.joke });
  } catch (err) {
    res.status(500).json({ greska: err.message });
  }
});
```

Provjera verzije

Pokreni `node --version`. Ako je 18 ili veća, `fetch` radi out-of-the-box. Ako je 16 ili manje — moraš instalirati `node-fetch` ili nadograditi Node.

npm i package.json: zavisnosti i skripte

npm (Node Package Manager) je alat koji dolazi sa Node-om. Služi za instaliranje paketa i upravljanje projektima.

Inicijalizacija novog projekta

```
npm init -y
# Pravi package.json sa default postavkama.
```

Instalacija paketa

```
npm install express
# ili krace: npm i express

# Vise paketa odjednom:
npm i express ejs better-sqlite3 bcryptjs

# Razvojne zavisnosti (samo za testiranje / dev):
npm i --save-dev nodemon

# Globalno (rijetko trebamo):
npm i -g typescript
```

Šta nastaje

- `node_modules/` — folder sa instaliranim paketima (NIKAD u git!)
- `package.json` — popis zavisnosti i meta-podaci
- `package-lock.json` — tačne verzije (commituj ga)

Skripte u package.json

```
{
  "name": "moj-projekat",
  "version": "1.0.0",
  "scripts": {
    "start": "node ./bin/www",
    "dev": "node --watch ./bin/www",
    "test": "echo 'jos nema testova' && exit 1"
  },
  "dependencies": {
    "express": "^5.0.0",
    "ejs": "^3.1.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.0"
  }
}

# Pokretanje skripti:
npm start          # ekvivalent: node ./bin/www
```

```
npm run dev      # ekvivalent: node --watch ./bin/www
npm test         # ekvivalent: definirana test komanda
```

Semver: ^, ~, tačno

- "**^5.0.0**" — bilo koja 5.x.x (ali ne 6.0.0)
- "**~5.0.0**" — bilo koja 5.0.x (ali ne 5.1.0)
- "**5.0.0**" — tačno ta verzija

.gitignore

Uvijek dodaj `node_modules` u `.gitignore`. To su stotine MB-a datoteka koje se mogu uvijek vratiti komandom `npm install`.

Praktični tipovi

node --watch (auto-restart pri promjeni)

Umjesto da stalno ručno restartuješ server, koristi `--watch`:

```
node --watch ./bin/www

# ili kao npm skripta:
# "dev": "node --watch ./bin/www"

# Svaki put kad sacuvas neki .js fajl, Node ce restartovati server.
# Radi od Node 18.11+. Za starije Node-ove postoji "nodemon" paket.
```

REPL (interaktivni JS terminal)

```
$ node
> const x = 5
> x * 2
10
> [1,2,3].map(n => n * 2)
[ 2, 4, 6 ]
> .exit
```

REPL je odličan za **brzo testiranje sintakse** bez pravljenja fajla. Probaj kad ne znaš da li nešto radi kako misliš.

.env fajl + dotenv

```
# Instaliraj:
npm i dotenv

# Napravi .env fajl (i dodaj ga u .gitignore!):
PORT=4000
SESSION_SECRET=tajni-kljuc-koji-niko-ne-zna
DB_HOST=localhost

# U app.js ili bin/www, učitaj sasvim na pocetku:
require("dotenv").config();

# Sada koristi:
const port = process.env.PORT || 3000;
const secret = process.env.SESSION_SECRET;
```

console.log triks za objekte

```
const obj = { a: 1, b: { c: 2, d: { e: 3 } } };

console.log(obj);
// { a: 1, b: { c: 2, d: [Object] } }  <- skraceno!

// Za PUNI ispis:
```

```
console.log(JSON.stringify(obj, null, 2));
// {
//   "a": 1,
//   "b": {
//     "c": 2,
//     "d": { "e": 3 }
//   }
// }

// Ili:
console.dir(obj, { depth: null });
```

Debug ruta

U Express ruti, dodaj `console.log(req.body)` da vidiš šta forma šalje. Ovo je 80% "debugovanja" na Express-u.

Završne misli i šta dalje

Ako razumiješ sintakse iz ovih 25 poglavlja, čitanje i pisanje Express koda postaje prirodno. Većina stvari na vježbama je kombinacija ovih osnovnih konstrukcija.

Šta vrijedi učvrstiti vježbom

- Destrukturiranje (`const { x, y } = obj`) — vidiš u svakoj ruti.
- Arrow funkcije — vidiš u svakoj middleware definiciji.
- `async/await` — koristit ćeš ga čim trebaš nešto da čekaš.
- `map/filter` — koristit ćeš ga čim trebaš transformisati podatke iz baze.

Korisni linkovi

- **MDN Web Docs** — developer.mozilla.org — najbolja referenca za JavaScript sintaksu.
- **Node.js docs** — nodejs.org/api — referenca za sve ugrađene module.
- **Express docs** — expressjs.com — referenca za rute, middleware.
- **npm** — npmjs.com — pretraga paketa.

Kratak "cheat sheet"

Stara sintaksa	Moderna sintaksa
<code>var x = 1;</code>	<code>const x = 1; / let x = 1;</code>
<code>function (x) { return x*2; }</code>	<code>x => x * 2</code>
<code>"Zdravo " + ime</code>	<code>`Zdravo \${ime}`</code>
<code>var a = obj.a; var b = obj.b;</code>	<code>const { a, b } = obj;</code>
<code>arr.concat(arr2)</code>	<code>[...arr, ...arr2]</code>
<code>if (x !== null) return x; else return 5;</code>	<code>x ?? 5</code>
<code>if (obj && obj.a && obj.a.b)</code>	<code>obj?.a?.b</code>
<code>.then(x => ...).then(y => ...)</code>	<code>const x = await ...; const y = await ...;</code>
<code>require('fs').readFileSync</code>	<code>await fs.promises.readFile</code>

Sretno na ostatku semestra! Ako nešto nije jasno — pitaj na vježbama.